MODERN APPLICATION DEVELOPMENT II · PROJECT STATEMENT

DemoDay Portal - V1

Hackathon demo slot booking & judging application

Roles: Admin · Judge · Participant **Stack:** Flask · VueJS · SQLite · Redis · Celery · Bootstrap **Version:** 1.0 · 21 Oct 2026

Hackathons and project expos need efficient systems to run demo day: organisers schedule demo rounds, judges announce when they are available, teams book a time to present, and judges score every team against the same rubric. Most organisers still juggle spreadsheets, forms, emails and chat groups, which makes it hard to track judge availability, prevent double bookings, collect scores consistently and publish results on time.

You are required to build a **DemoDay Portal** web application that allows **Admin**, **Judges** and **Participants** to interact with the system based on their roles.

Frameworks to be used

These are the mandatory frameworks on which the project has to be built.

- Flask for API
- VueJS for UI
- VueJS Advanced with CLI (only if required, not necessary)
- Jinja2 templates if using CDN, only for the entry point (not to be used for UI)
- Bootstrap for HTML generation and styling (no other CSS framework is allowed)
- SQLite for database (no other database is permitted)
- Redis for caching
- Redis and Celery for batch jobs

> **Note:** All demos should be possible on your local machine. Any other framework/library is not allowed apart from those given above. Supporting extensions of these frameworks (for example Flask-SQLAlchemy, Flask-Security or Flask-JWT-Extended, Flask-Caching, Vue Router and ChartJS) may be used. The database must be created programmatically (via table creation or model code). Manual database creation (e.g. DB Browser for SQLite) is NOT allowed.

Roles & Functionalities

This platform has three roles:

Admin

Admin is the pre-existing superuser of the application.

- Can create, update, archive and remove hackathon tracks.
- Can create, update and manage demo events under a track.
- Can configure event timelines (slot creation and booking windows).
- Can view all judge slots and all bookings.
- Can reschedule a booking to another slot in exceptional situations.
- Can change the judge assigned to a slot if required.
- Can deactivate or blacklist participants and judges.

- Can open and close judge slot creation.
- Can open and close participant slot booking.
- Can create and manage the evaluation rubric of an event.
- Can add and manage judges and assign them to events.
- Can search participants, judges, events and bookings.
- Can publish results of an event.
- Can view reports and statistics.

Judge

- Can log in only after being created by Admin.
- Can view events assigned to them.
- Can create demo slots during the slot creation period.
- Can update or remove slots before booking begins (and only while nobody has booked them).
- Can view teams booked into their slots.
- Can evaluate teams using the event rubric.
- Can submit marks and remarks.
- Can mark a slot as completed.

Participant

- Can register, log in and update their profile.
- Can view events that are open or upcoming.
- Can search and filter events by track, level, mode and name/venue.
- Can book an available demo slot with a project title.
- Can view booking status.
- Can cancel a booking before the booking deadline.
- Can view demo history.
- Can view published results and judge feedback.

Key Terminologies

Admin: A user with the highest level of access who manages the entire demo day.

Participant: A registered user (team representative) who books a slot and presents a project.

Judge: A mentor or industry expert who offers demo slots and evaluates teams.

Track: A theme of the hackathon (e.g. AI / ML, Web Apps, IoT) under which demo events run.

Track

A hackathon theme grouping one or more demo events.

- Track ID
- Track Code (unique)
- Track Name
- Description
- Status (Active / Archived)

Demo Event

A demo round created by the Admin under a track.

- Event ID, Track ID, Event Name
- Mode (Online / Offline), Venue
- Level (Beginner / Intermediate / Advanced)
- Demo Duration (minutes), Maximum Score, Instructions
- Slot Creation Start / End Date
- Slot Booking Start / End Date
- Status (Draft / Slot Creation / Booking Open / Closed / Completed)
- Results Published (Yes / No)

Evaluation Rubric (Criterion)

One criterion judges use to score a demo. Weightages of an event add up to 100%.

- Criterion ID, Event ID
- Criterion Name
- Maximum Marks
- Weightage (%)
- Description

Demo Slot

A time window offered by a judge for a particular event.

- Slot ID, Event ID, Judge ID
- Date, Start Time, End Time
- Maximum Team Capacity, Booked Count
- Available Seats (capacity − booked)
- Status (Available / Full / Cancelled / Completed)
- Meeting Link (required for online events)

Booking

A record of a participant booking a demo slot.

- Booking ID, Participant ID, Slot ID, Event ID
- Project Title
- Booking Date
- Status (Booked / Cancelled / Completed)

Evaluation

The judge's assessment of one booking.

- Evaluation ID, Booking ID
- Participant ID, Judge ID
- Total Score
- Remarks
- Evaluation Date, Status

Criterion Score

Marks given for one rubric criterion inside one evaluation.

- Score ID
- Evaluation ID
- Criterion ID
- Marks

Judge Profile

Details of a registered judge.

- Judge ID, Name
- Contact Details (email, phone)
- Organization, Designation
- Expertise
- Status (active / inactive / blacklisted)

Note: The above tables and fields are not exhaustive. You can add more or fewer tables and fields as required. All three roles must share one unified user table that differentiates them by role. A judge can be assigned to many events and an event can have many judges.

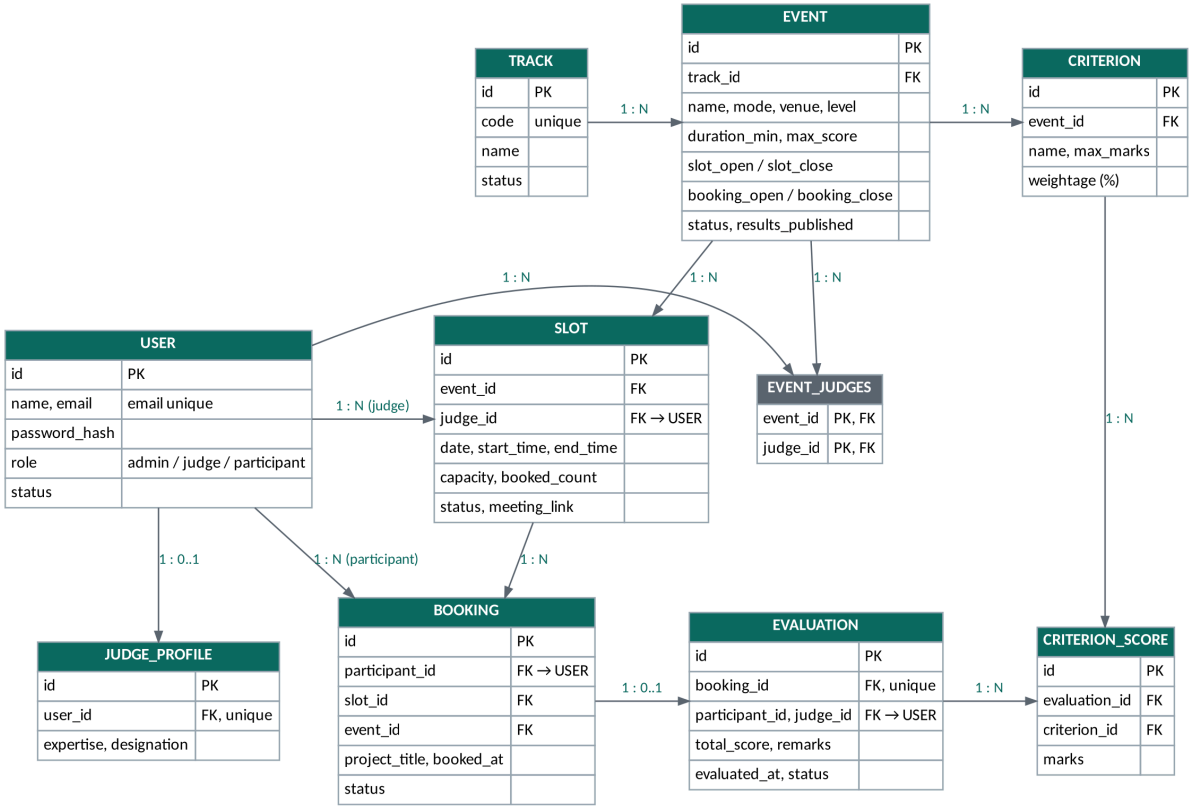
Demo Event Lifecycle

Every demo event moves through the statuses below. The Admin changes the status; each status decides what judges and participants can do.

Status	Set by Admin when	What is allowed
Draft	The event is created	Admin edits event details, timeline and rubric. Hidden from participants.
Slot Creation	At least one judge is assigned	Assigned judges create, edit and remove their slots within the slot creation dates. Participants can view the event but not book.
Booking Open	Slots exist and rubric weightages total 100%	Participants book one slot per event and may cancel until the booking end date. Judges can no longer change slots that have bookings.
Closed	The booking window is over	No new bookings or cancellations. Judges evaluate booked teams on or after the demo date.
Completed	All demos are held	Admin publishes results once no booking is waiting for evaluation; participants then see marks and remarks.

Suggested Data Model

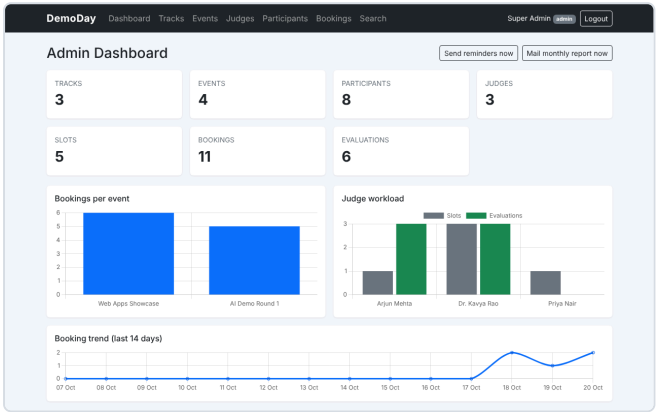
An indicative ER diagram for the entities above. PK = primary key, FK = foreign key. Only the main columns are shown; you may add or remove tables and fields as required.



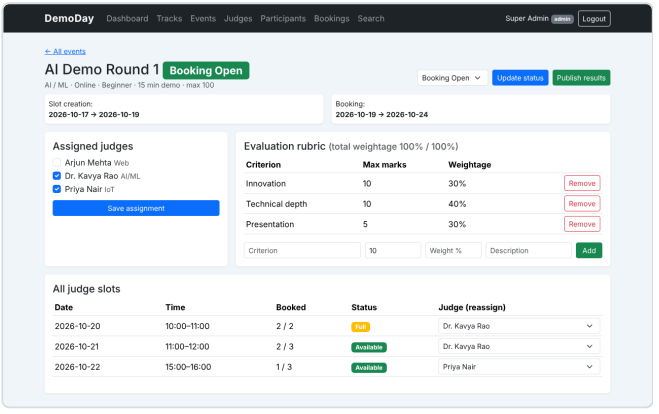
Application Wireframe

The sample screens below illustrate the application's flow and navigation for each role, shown with sample data.

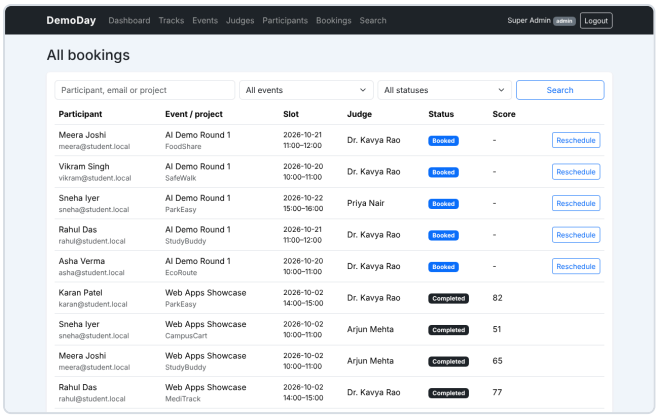
> **Note:** Replication of the exact UI is NOT mandatory. You are encouraged to design your own UI while keeping the intended functionality and flow.



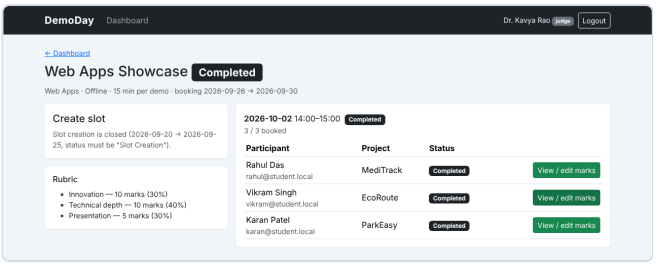
Admin dashboard: totals, bookings per event, judge workload and booking trend.



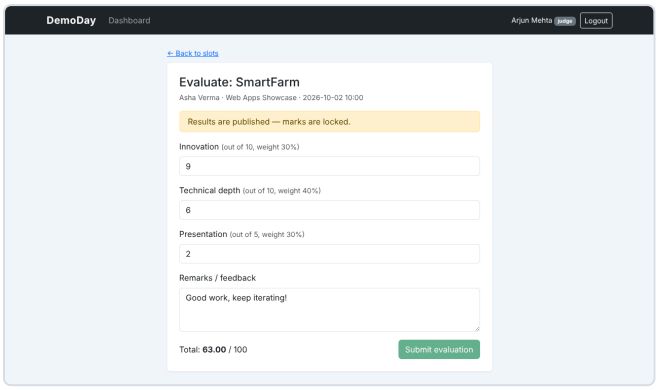
Event detail (Admin): status, judge assignment, rubric, all judge slots with reassignment, publish results.



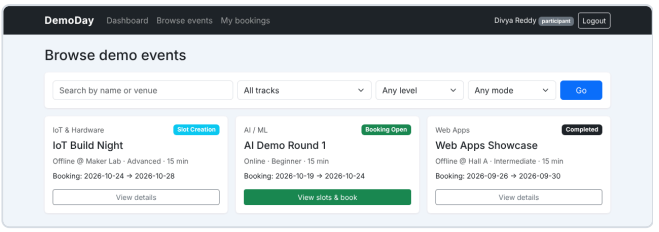
All bookings (Admin): search, filter by event and status, reschedule a booking.



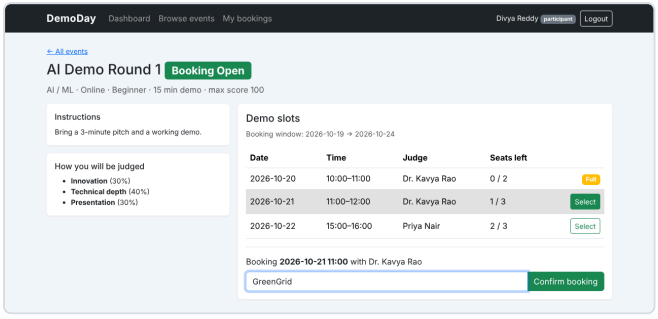
Judge: slots for an event and the teams booked into each.



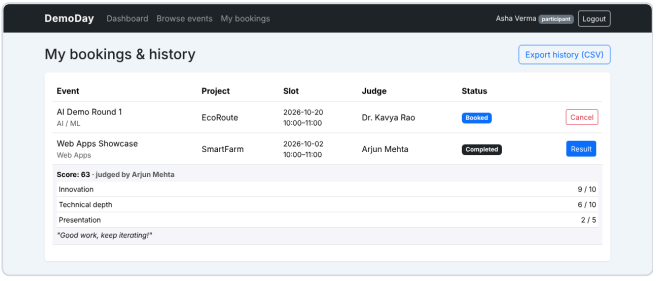
Judge: rubric scoring form with a live weighted total.



Participant: search and filter events.



Participant: choose a slot and confirm the booking.



Participant: booking history, cancellation, result with judge feedback, CSV export.

Similar Apps

- Devpost
- Devfolio
- HackerEarth
- Unstop
- Calendly (slot booking)

Core Features

Authentication

- Login/Register for Participants.
- Login only for Admin and Judges (no registration).
- Only one Admin must exist and should be created programmatically.
- Role-based access control and authentication using Flask Security (session/token) or JWT based token.

- A unified user model to differentiate all types of user roles.
- Inactive or blacklisted users must not be able to log in or use the APIs.

Admin Functionalities

Admin dashboard showing:

- Total number of tracks and events.
- Total number of participants and judges.
- Total number of demo slots.
- Total number of bookings and evaluations.

Admin must pre-exist and be created programmatically after database creation. [No admin registration allowed]

Admin can:

- Create and manage tracks and events.
- Create the evaluation rubric of an event.
- Add new judges and assign them to events.
- Open and close judge slot creation.
- Open and close participant booking.
- View all judge slots and all bookings.
- Search participants, judges, events and bookings.
- Reschedule bookings when required.
- Reassign a slot to another judge if necessary.
- Deactivate or blacklist users.
- Publish results.

Judge Functionalities

Judge dashboard should display:

- Assigned events.
- Created demo slots.
- Number of booked teams.
- Pending evaluations.

Judge can:

- Create demo slots.
- Update slots before booking starts.
- View booked teams.
- Evaluate teams using the rubric.
- Submit marks and remarks.
- Mark a slot as completed.

Participant Functionalities

Participants must self-register and log in. Participant dashboard should display:

- Events open for booking (with seats left).
- Upcoming demos.
- Booked demo slots.
- Published results.

Participants can:

- Book demo slots.
- Cancel bookings before the deadline.
- View booking history and status.
- View judge remarks.
- Edit profile.
- Search events.

Backend Jobs (Using Celery)

a. Scheduled Job – Daily Demo Reminder

The application should send daily reminders to participants on Google Chat using webhooks, SMS or email.

- Send reminders to participants whose demo is today or tomorrow, containing: event name, project title, date and time, assigned judge, meeting link or venue, and event instructions.
- Runs daily at a chosen time.

b. Scheduled Job – Monthly Activity Report

Devise a monthly report for the Admin, created using HTML and sent via mail.

- Generate monthly statistics including: number of events conducted, number of participants evaluated, number of completed bookings, pending evaluations, per-event bookings and average score, and the most active judges.
- Generated on the first day of every month.
- Sent to Admin via email (HTML or PDF).

c. User Triggered Async Job – Export Demo History as CSV

- Participants can export their demo history.
- CSV should include: Participant ID, Event Name, Track, Demo Date, Time, Judge, Project Title, Score (only after results are published), Booking Status.
- Triggered from the participant's bookings page.
- This should trigger a batch job and send an alert once completed.

Performance and Caching

- Use Redis caching for: the event catalogue and search results, event details including the rubric, event schedules (slots and seats left), and the list of active tracks.
- Implement cache expiry.
- Invalidate the affected cache entries whenever events, rubrics, slots or bookings change.
- Optimize API response times.

Other Core Functionalities

- Judges can create slots only during the slot creation period, and only for events they are assigned to.
- A judge's slots must not overlap, and each slot must be at least as long as the event's demo duration.
- Slot creation can open only after at least one judge is assigned; booking can open only when slots exist and rubric weightages total 100%.
- Participants can book slots only during the booking period.
- Participants can book only one slot per event.
- Prevent overbooking beyond slot capacity.
- Automatically mark slots as Full when capacity is reached, and Available again after a cancellation.
- Judges can evaluate only teams booked into their own slots.
- The rubric is locked once any evaluation exists.
- Results can be published only when no booking is waiting for evaluation; marks stay hidden from participants until then.
- Admin can reschedule bookings and change the judge of a slot.
- Prevent duplicate bookings for the same event.
- Maintain complete booking and evaluation history.
- Allow searching and filtering of events.
- Allow Admin to view all historical data.

Recommended and/or Optional Functionalities

- Well-designed PDF reports for the monthly activity report (you can choose between HTML and PDF
- Frontend validation on all form fields using HTML5 validation or JavaScript.

reports).
- Charts for analytics (bookings per event, judge workload, booking trends) using ChartJS.
- Single responsive UI for both mobile and desktop.
- Unified UI that works across devices.
- Add to desktop feature.
- Backend validation within your APIs.
- Styling and aesthetics using Bootstrap.
- Notifications for successful booking or cancellation.
- Email notifications on result publication.
- Downloadable scorecard; any additional feature relevant to demo days.

Possible Folder Structure

An indicative layout for the project. You may organise your code differently.

```
demoday/
├── app.py            app factory, admin seeding
├── config.py         settings (DB, JWT, Redis, Celery, mail)
├── extensions.py     db, jwt, cache objects
├── models.py         all tables
├── utils.py          role_required, validators
├── cache_utils.py    Redis cache helpers
├── celery_app.py     Celery + beat schedule
├── tasks.py          background jobs
├── mailer.py         SMTP + Google Chat webhook
├── requirements.txt
├── README.md
├── routes/
│   ├── auth.py       admin.py     judge.py
│   └── events.py     participant.py
├── templates/
│   ├── index.html    Vue entry (Jinja)
│   └── mail/         email templates
└── static/
    ├── css/style.css
    └── js/           app.js, router.js, api.js,
                      components/, pages/
```